

Parte 2: Introducción a Wireshark

Redes de Computadoras - CC308

José Antonio Mérida Castejón - 201105

13 de julio de 2026

Introducción

En la primera parte de este laboratorio se replicaron esquemas de comunicación previos a las redes modernas (código Morse, código Baudot y un protocolo propio de conmutación de mensajes), lo cual permitió evidenciar de forma práctica los problemas de fiabilidad, ergonomía y enrutamiento que enfrentaban dichos esquemas. En esta segunda parte se aborda una herramienta central para el estudio de las redes de computadoras actuales: Wireshark, un analizador de protocolos que permite capturar y examinar el tráfico que circula por una interfaz de red en tiempo real.

El objetivo de esta parte es familiarizarse con el entorno de Wireshark, personalizando perfiles, layout y filtros, para luego realizar una captura de paquetes sobre la interfaz de red activa de la computadora y analizar una solicitud HTTP real, identificando las versiones del protocolo utilizadas, los encabezados relevantes de la solicitud y de la respuesta, y discutiendo consideraciones de rendimiento y seguridad relacionadas con el uso de esta herramienta.

Personalización del Entorno

Configuración de perfil

Se creó un nuevo perfil en Wireshark para personalizar la captura sin afectar la configuración por defecto.

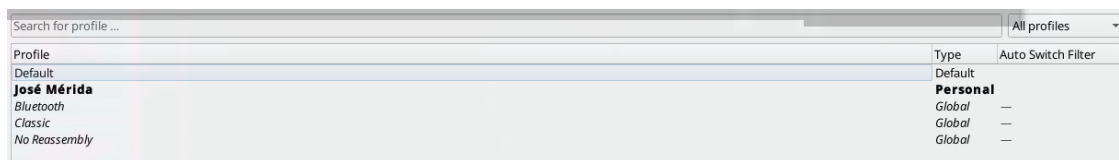


Figura 1: Creación del perfil personalizado en Wireshark.

Layout, Filtro y Botón de Filtro

Se modificó el layout de la interfaz, agregando un botón de filtro y una regla de coloreo personalizada.

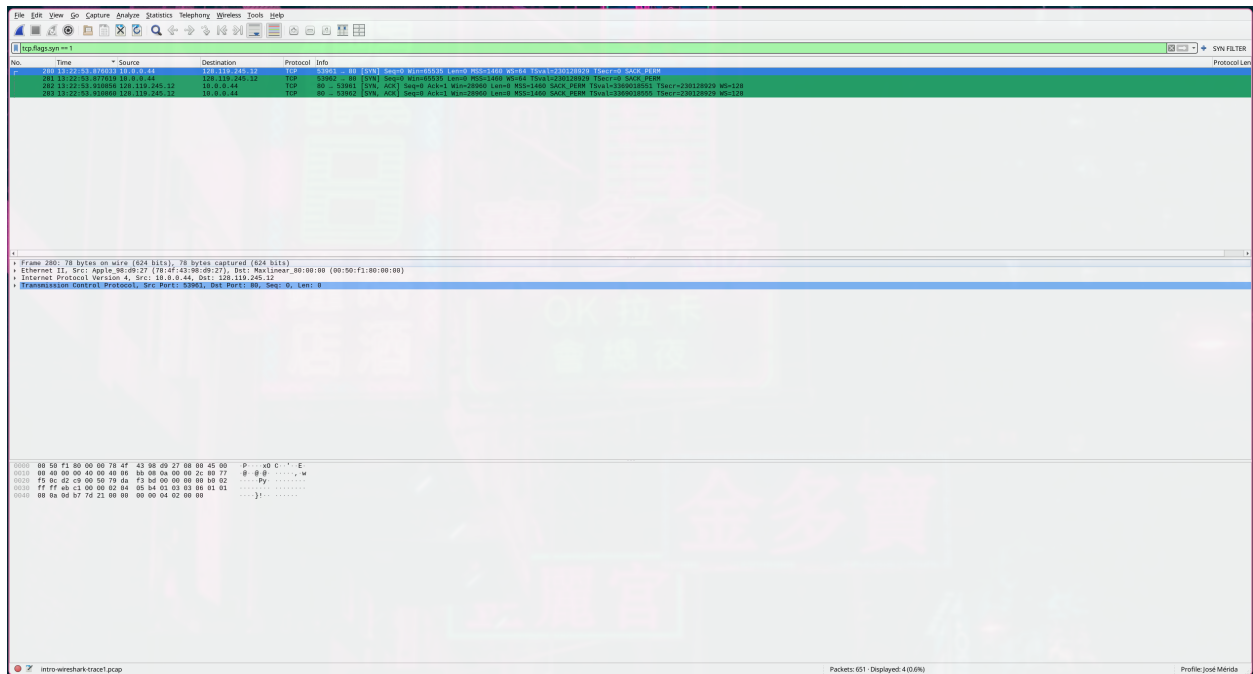


Figura 2: Cambio de layout, botón agregado y regla de coloreo.

Interfaces Virtuales

Se removieron las interfaces virtuales o que no se encuentran en uso dentro del sistema, esto también se verifica por medio del comando `ifconfig` ejecutado en una sección posterior del laboratorio.

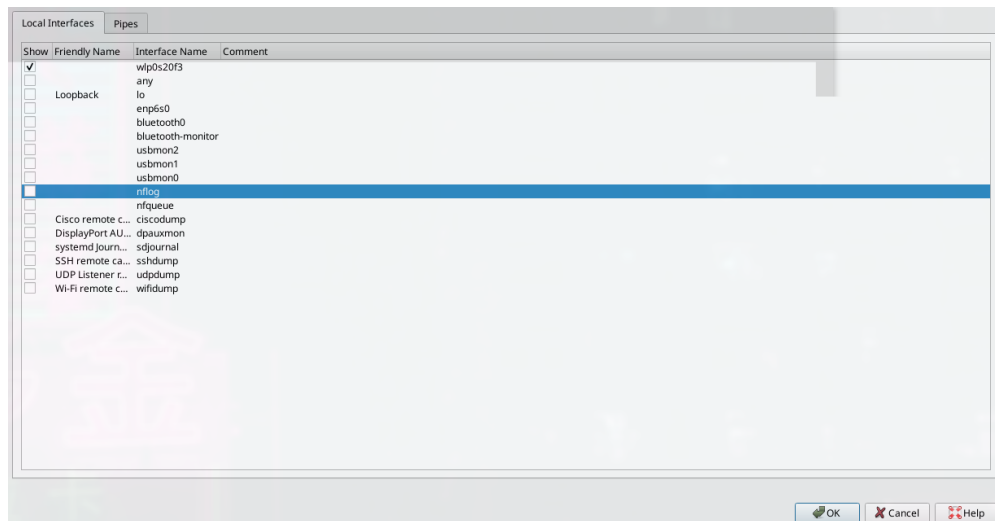


Figura 3: Listado de interfaces sin las interfaces virtuales.

Configuración de la Captura de Paquetes

```
> ifconfig
enp6s0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
    ether 08:bf:b8:a0:11:50 txqueuelen 1000 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
    device memory 0x82f00000-82ffffff

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 1141 bytes 137838 (134.6 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 1141 bytes 137838 (134.6 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

wlp0s20f3: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.1.17 netmask 255.255.255.0 broadcast 192.168.1.255
    inet6 fe80::aa69:feb:4926:90d1 prefixlen 64 scopeid 0x20<link>
    inet6 2800:98:1124:aa7::1 prefixlen 128 scopeid 0x0<global>
    inet6 2800:98:1124:aa7:ff97:6b9e:b3c6:6e5a prefixlen 64 scopeid 0x0<global>
    ether 0a:62:e7:72:da:65 txqueuelen 1000 (Ethernet)
    RX packets 34769 bytes 34577387 (32.9 MiB)
    RX errors 0 dropped 1 overruns 0 frame 0
    TX packets 15481 bytes 7619843 (7.2 MiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

Figura 4: Ejecución de comando ifconfig.

El comando `ifconfig` muestra todos los adaptadores de red disponibles en la computadora, al igual que sus direcciones IP (en caso de tener una asignada) e ID's de hardware. En este caso podemos ver tres dispositivos:

- **enp6s0**: Interfaz de ethernet, no tiene IP asignada y no ha mandado o recibido ningún paquete. Esto es consistente con el hecho que no está conectado ningún cable de ethernet a la computadora al momento de ejecutar el comando.
- **lo**: Tiene asignada la IP `127.0.0.1`, es una interfaz virtual utilizada por el sistema operativo. Este es el localhost.
- **wlp0s20f3**: Interfaz de internet inalámbrica, se puede ver la IP asignada `192.168.1.17` al igual que los paquetes transmitidos (TX) y paquetes recibidos (RX) desde el momento de arranque. Esta es la interfaz de red activa del sistema.

Utilizando el dispositivo `wlp0s20f3` se realizó la captura con la siguiente configuración:

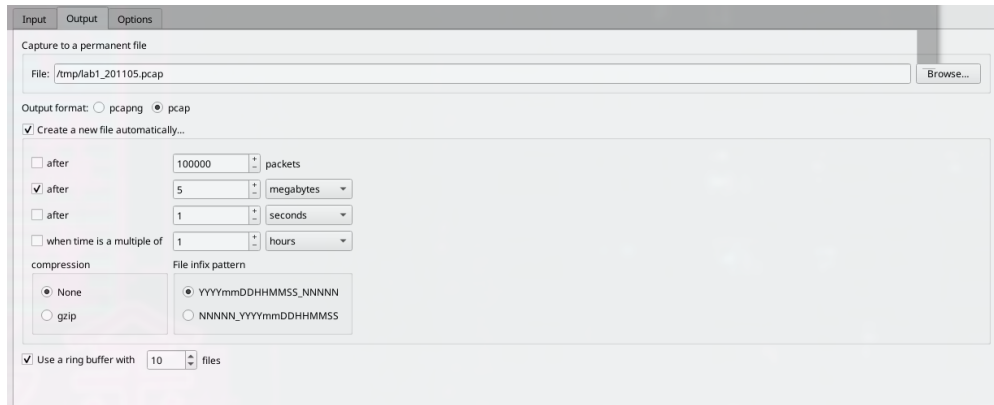


Figura 5: Configuración de las opciones de salida de la captura en Wireshark.

Los archivos resultantes se encuentran dentro del repositorio de la entrega, dentro del directorio `files`.

Análisis de Paquetes

Para el análisis se filtró el tráfico capturado utilizando el filtro `http`, aislando así únicamente las solicitudes y respuestas del protocolo HTTP generadas al cargar la página de prueba del laboratorio (`gaia.cs.umass.edu`).

No.	Time	Source	Destination	Protocol	Length	Info
100	10:15:11.044318484	192.168.1.17	192.168.1.11	HTTP	520	GET /gaia.cs.umass.edu/ HTTP/1.1
101	10:15:11.044318484	192.168.1.11	192.168.1.17	HTTP	507	200 OK (text/html)
102	10:15:11.044318484	192.168.1.17	192.168.1.11	HTTP	476	GET /gaia.cs.umass.edu/ HTTP/1.1
103	10:15:11.044318484	192.168.1.11	192.168.1.17	HTTP	648	200 OK (text/html)

Figura 6: Solicitudes y respuestas HTTP capturadas al filtrar por `http`.

Preguntas

- ¿Qué versión de HTTP está ejecutando su navegador?
HTTP/1.1, evidenciado por la solicitud GET inicial hacia la página.
- ¿Qué versión de HTTP está ejecutando el servidor?
HTTP/1.1, evidenciado por la respuesta 200 OK del servidor.
- ¿Qué lenguajes (si aplica) indica el navegador que acepta a el servidor?

```

GET /wireshark-labs/INTRO-wireshark-file1.html HTTP/1.1\r\n
Host: gaia.cs.umass.edu\r\n
Connection: keep-alive\r\n
Upgrade-Insecure-Requests: 1\r\n
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/142.0.0.0 Safari/537.36\r\n
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7\r\n
Accept-Encoding: gzip, deflate\r\n
Accept-Language: en-US,en;q=0.9\r\n
\r\n
[Response in frame: 106]
[Full request URI: http://gaia.cs.umass.edu/wireshark-labs/INTRO-wireshark-file1.html]

```

Figura 7: Encabezados de la solicitud GET, incluyendo `Accept-Language`.

en-US, en. Inglés Americano e Inglés.

- d. ¿Cuántos bytes de contenido fueron devueltos por el servidor?

```

Hypertext Transfer Protocol
  HTTP/1.1 200 OK\r\n
  Date: Mon, 13 Jul 2026 22:15:31 GMT\r\n
  Server: Apache/2.4.62 (AlmaLinux) OpenSSL/3.5.5 mod_fcgid/2.3.9 mod_perl/2.0.12 Perl/v5.32.1\r\n
  Last-Modified: Tue, 28 Oct 2025 05:59:01 GMT\r\n
  ETag: "51-64231b6715777"\r\n
  Accept-Ranges: bytes\r\n
  Content-Length: 81\r\n
    [Content length: 81]
  Keep-Alive: timeout=5, max=100\r\n
  Connection: Keep-Alive\r\n
  Content-Type: text/html; charset=UTF-8\r\n
  \r\n
  [Request in frame: 102]

```

Figura 8: Encabezados de la respuesta HTTP, incluyendo `Content-Length`.

81 bytes.

- e. En el caso que haya un problema de rendimiento mientras se carga la página, ¿en qué elementos de la red convendría escuchar los paquetes?

Convendría capturar en los distintos puntos del recorrido entre el cliente y el servidor: en el host del cliente (para descartar problemas locales de resolución DNS o de la pila TCP/IP), en el gateway o router de salida de la red local, y, de ser posible, en algún punto cercano al servidor o en el servidor mismo. Esto permite comparar los tiempos de ida y vuelta (RTT) en cada segmento de la ruta y aislar si la latencia se origina en la red del cliente, en el tránsito por Internet, o en el procesamiento del servidor [1]. Por ejemplo, si el retardo entre el SYN y el SYN-ACK ya es alto capturando en el cliente, el problema probablemente sea de red; si en cambio el *handshake* es rápido pero la respuesta HTTP tarda en llegar después del ACK, el cuello de botella está del lado del servidor.

- f. ¿Es conveniente instalar Wireshark en el servidor? Justifique

En general no es recomendable instalar y ejecutar Wireshark directamente en un servidor de producción. Capturar y analizar tráfico consume recursos de CPU, memoria y disco (especialmente si el servidor maneja un volumen alto de conexiones), lo cual puede degradar el rendimiento del servicio que se está intentando diagnosticar. Además, una captura completa en el servidor expone tráfico de todos los clientes que se conectan a él, incluyendo datos potencialmente sensibles, lo que representa un riesgo de seguridad y privacidad si esa captura

no se maneja con cuidado [2]. Por estas razones, generalmente se prefiere capturar tráfico de forma más selectiva y en un punto intermedio de la red (como un *switch* con puerto espejo o *port mirroring*), o bien utilizar la captura en el servidor solo de forma puntual, por tiempo limitado y con filtros específicos, en vez de dejarla corriendo de manera permanente.

Discusión General

A lo largo de las dos partes de este laboratorio se recorrió el problema de la comunicación en redes desde dos extremos distintos. En la primera parte se trabajó con esquemas de comunicación manuales y de bajo nivel (Morse, Baudot y un protocolo de conmutación diseñado por el grupo), lo que permitió notar de primera mano cómo la ausencia de mecanismos de verificación y corrección de errores hace que la comunicación sea poco confiable, incluso entre pocos participantes y con mensajes cortos. En esta segunda parte, en cambio, se utilizó Wireshark para observar cómo estos mismos problemas (fiabilidad, negociación de conexión, direccionamiento) están resueltos en la práctica por protocolos modernos como TCP y HTTP, mediante mecanismos como el *three-way handshake*, los números de secuencia, y encabezados estandarizados que describen el contenido de cada mensaje.

Esta conexión resulta particularmente clara al comparar el protocolo de conmutación propuesto en la primera parte con el comportamiento observado en la captura de HTTP. El intercambio LISTO?/OK diseñado por el grupo para evitar que un emisor transmitiera hacia un conmutador ocupado es conceptualmente similar al *handshake* SYN/SYN-ACK/ACK que utiliza TCP antes de permitir el envío de datos, y la etiqueta EMISOR | RECEPTOR utilizada para el enrutamiento cumple un rol análogo al de las direcciones IP y los puertos de origen y destino visibles en cada paquete capturado. Ver estos mecanismos implementados y funcionando en una herramienta como Wireshark ayudó a entender por qué las redes modernas necesitaron formalizar, en protocolos como TCP/IP y HTTP, muchas de las decisiones de diseño que el grupo tuvo que tomar de forma artesanal durante la primera parte del laboratorio.

Conclusiones

- Personalizar el entorno de Wireshark (perfiles, layout, filtros y reglas de coloreo) antes de capturar tráfico facilita significativamente el análisis posterior, al permitir aislar rápidamente el tráfico relevante entre el volumen de paquetes que normalmente circula por una interfaz de red activa.
- El comando `ifconfig` y la ventana de interfaces de Wireshark evidenciaron que, más allá de la interfaz de red física en uso, el sistema mantiene numerosas interfaces virtuales; identificarlas y filtrarlas es un paso necesario para capturar únicamente el tráfico de interés.
- El análisis de una solicitud HTTP concreta permitió confirmar de forma práctica conceptos normalmente vistos solo de forma teórica, como la versión del protocolo utilizada por cliente y servidor, los encabezados de negociación de contenido (`Accept-Language`) y el tamaño exacto del contenido devuelto (`Content-Length`).
- Diagnosticar problemas de rendimiento en una red requiere capturar tráfico en múltiples puntos de la ruta cliente-servidor, y no únicamente en un extremo, para poder aislar en qué segmento se origina la latencia.

- Herramientas de análisis de tráfico como Wireshark son poderosas mientras se usan de forma responsable: instalarlas y dejarlas corriendo en un servidor de producción introduce riesgos de rendimiento y de seguridad que generalmente no justifican su conveniencia frente a alternativas como la captura selectiva o el uso de un puerto espejo en un punto intermedio de la red.
- En conjunto, ambas partes del laboratorio mostraron cómo los problemas fundamentales de la comunicación (fiabilidad, negociación de conexión y direccionamiento) se mantienen constantes desde los esquemas más rudimentarios hasta los protocolos que sostienen las redes modernas, cambiando únicamente el nivel de formalización y automatización con el que se resuelven.

Referencias

- [1] Wireshark Foundation, *Wireshark User's Guide: Chapter 7. Analyzing Protocols*. [En línea]. Disponible en: https://www.wireshark.org/docs/wsug_html_chunked/. [Accedido: 13-jul-2026].
- [2] SANS Institute, *Packet Capture Best Practices: Security and Performance Considerations*. [En línea]. Disponible en: <https://www.sans.org/white-papers/>. [Accedido: 13-jul-2026].
- [3] GeeksforGeeks, *ifconfig command in Linux with Examples*. [En línea]. Disponible en: <https://www.geeksforgeeks.org/linux-unix/ifconfig-command-in-linux-with-examples/>. [Accedido: 13-jul-2026].
- [4] Stack Overflow, *How to check length of protocols*. [En línea]. Disponible en: <https://stackoverflow.com/questions/24197371/how-to-check-length-of-protocols>. [Accedido: 13-jul-2026].